


**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY**

**VIZIANAGARAM**

**VIZIANAGARAM – 535003**

**ANDHRA PRADESH**

# DISASTER ALERT APP

*By*

**SK.B.JANI BASHA**

**(Regd. No. 25VV5A1274)**

*Under the supervision of*

**MRS. MADHUMITA CHANDA**

**MRS.M.GEETHA MADHURI YADAV**

Department of Information Technology

Viziangaram – 535003

**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY**

**VIZIANAGARAM**

**VIZIANAGARAM – 535003**

**ANDHRA PRADESH**

**Regd.No: 25VV5A1274**

This is to certify that this is a bonafide record of practical work done by **SHAIK BELLAMKONDA JANI BASHA** of III B.Tech I Semester Class in **USER INTERFACE DESIGN USING FLUTTER** Lab during the year 2026-27.
No.of Tasks Completed and Certified:

**Lecture In-Charge** **Head of The Department**

**Date:**

# DEPARTMENT OF INFORMATION TECHNOLOGY
# JNTU-GURAJADA VIZIANAGARAM
# COLLEGE OF ENGINEERING VIZIANAGARAM (A)
# VIZIANAGARAM

Website:www.jntugvcev.edu.in

**Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01**

**Academic Year: 2026** **Regulation: R23**

## COURSE OUTCOMES

| NBA Subject Code | Course Outcomes | |
|---|---|---|
| R233112ES01 | CO1 | Learns to Implement Flutter Widgets and Layouts. |
| | CO2 | Understands Responsive UI Design and with Navigation in Flutter. |
| | CO3 | Knowledge on Widgets and customize widgets for specific UI elements, Themes |
| | CO4 | Understand to include animation apart from fetching data. |

## CO-PO Mapping

**Mapping of Course Outcomes (COs) with Program Outcomes (POs)**

| Course Outcomes | | Program Outcomes (POs) | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | PO 1 | PO 2 | PO 3 | PO 4 | PO 5 | PO 6 | PO 7 | PO 8 | PO 9 | PO 10 | PO 11 | PO 12 | PSO 1 | PSO 2 | PSO 3 |
| R233112ES01 | CO1 | 3 | 2 | 3 | 2 | 3 | 1 | 1 | 1 | 2 | 2 | 1 | 2 | 3 | 3 | 2 |
| | CO2 | 3 | 3 | 3 | 2 | 3 | 1 | 2 | 2 | 2 | 3 | 2 | 2 | 3 | 3 | 2 |
| | CO3 | 3 | 2 | 3 | 2 | 3 | 2 | 1 | 1 | 2 | 2 | 1 | 2 | 3 | 2 | 2 |
| | CO4 | 3 | 3 | 3 | 3 | 3 | 2 | 1 | 2 | 3 | 3 | 2 | 3 | 3 | 3 | 3 |

**Enter correlation levels 1,2 and 3 as defined below:**

**1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"**

**Signature of the Course Instructor**

# INDEX


| S.No. | Date | Experiment Name | Page No. | Marks | Signature |
|---|---|---|---|---|---|
| 1 | 01-07-2026 | (a) Introduction<br>(b) Project Structure | 1-2 | | |
| 2 | 08-07-2026 | (a) SQlite Database Setup<br>(b) Installation of Flutter (with PATH Setup) | 3-7 | | |
| 3 | 22-07-2026 | (a) Installation of Dart SDK in Windows<br>(b) Installing Visual Studio Code | 8-9 | | |
| 4 | 05-08-2026 | (a) Creating a New Flutter Project<br>(b) Dart Basic Programming | 10-13 | | |
| 5 | 12-08-2026 | (a) Case Study of Disaster Alert App<br>(b) Exploring Flutter Widgets | 14-22 | | |
| 6 | 19-08-2026 | (a) Responsive UI Design<br>(b) Navigation Between Screens | 23-26 | | |
| 7 | 02-09-2026 | (a) Stateful and Stateless Widgets<br>(b)Custom Widgets and Styling | 27-31 | | |
| 8 | 16-09-2026 | (a)Forms and Validation<br>(b) Animations | 32-44 | | |
| 9 | 30-09-2026 | (a) REST API Integration<br>(b)Testing and Debugging | 45-46 | | |
| 10 | 07-10-2026 | (a) Work Flow Of Screens<br>(b) Conclusion | 47-51 | | |

# 1 Introduction

Flutter is an open-source software development framework created by Google. It allows developers to build high-quality, natively compiled applications for mobile, web, and desktop from a single codebase.

## Advantages of Flutter

- Cross-platform development with a single codebase.
- Hot reload for instant updates.
- Rich set of customizable widgets.
- High performance due to AOT compilation.
- Backed by Google and strong community support.

## Dart Language

Dart is the programming language used by Flutter. It is object-oriented, class-based, and supports JIT and AOT compilation. Dart syntax is simple and similar to Java/C++.

## Advantages of Dart

- Easy-to-learn syntax.
- Null safety to avoid runtime errors.
- Strong typing and async/await support.
- JIT for fast development, AOT for optimized production.

# Key Features of Flutter

- **Cross-Platform Development:** One codebase can run on multiple platforms such as Android, iOS, Web, Windows, macOS, and Linux.
- **Hot Reload:** Instantly view code changes without restarting the app, improving developer productivity and speeding up testing.
- **Widget-Based Framework:** Everything in Flutter is a widget — layouts, buttons, text, and images, which makes UI creation simple and flexible.
- **High Performance:** Flutter apps are compiled to native ARM code, ensuring fast performance and smooth animations.
- **Custom UI Design:** Allows the creation of flexible and visually appealing interfaces using Material Design (for Android) or Cupertino (for iOS) widgets.

# 2 Project Structure

The Disaster Alert App provides disaster information and emergency support through a critical-alert dashboard, hazard guidance, weather forecasts, live alerts, a map, alert details, SOS, emergency contacts, and safety settings.

**Representative Flutter project organization based on the screens :**

```
disaster_alert_app/
└── lib/
    ├── data/
    |   └── disaster_data.dart
    ├── models/
    |   └── alert.dart
    ├── screens/
    |   ├── home_screen.dart
    |   ├── survival_guide_screen.dart
    |   ├── earthquake_guide_screen.dart
    |   ├── weather_screen.dart
    |   ├── live_map_screen.dart
    |   ├── live_alerts_screen.dart
    |   ├── alert_details_screen.dart
    |   ├── sos_screen.dart
    |   ├── settings_screen.dart
    |   └── contacts_screen.dart
    ├── services/
    |   ├── database_helper.dart
    |   ├── weather_service.dart
    |   └── alert_service.dart
    ├── widgets/
    |   ├── alert_card.dart
    |   ├── hazard_tile.dart
    |   └── emergency_button.dart
    └── main.dart
```

## Description of Project Modules

- **lib/ – Typical Flutter source directory; source files were not supplied.**
- **data/ – May contain hazard categories and sample information; implementation is unverified.**
- **models/ – May define alert, weather, and emergency-contact records; illustrative only.**
- **screens/ – Visible screens include Dashboard, Survival Guide, Weather, Live Map, Live Alerts, Alert Details, SOS, Settings, and New Contact.**

# 3 SQLite Database Setup

SQLite is a lightweight relational database that stores application data locally on the device. In Flutter, the sqflite package provides access to SQLite databases. In the Disaster Alert App, SQLite can be used to store emergency contacts, saved alert records, and user preferences locally. The following steps describe the basic setup.

### 1.Add SQLite Dependencies:

– Open the pubspec.yaml file.
– Add the sqflite and path packages under dependencies.
• SQLite package configuration in pubspec.yaml

### 2. Install the Packages:

– Save the pubspec.yaml file.
– Run flutter pub get from the project terminal.
– Confirm that the packages are resolved before importing them into Dart files.

### 3. Create a Database Helper:

– Create a database_helper.dart file to open the local database and define its tables:

**SQLite database helper implementation**.

dependencies: sqflite:
^2.3.3+1 path: ^1.9.0
– Run flutter pub get to install the packages.

### 4. Initialize the Database:

– Open the database before performing insert, update, query, or delete operations:

```
final dbPath = await getDatabasesPath();
final path = join(dbPath,
'disaster_alert.db'); final db = await
openDatabase(path, version: 1,
onCreate: (db, version) async { await
db.execute('CREATE TABLE contacts(id
INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT, phone
TEXT, relationship TEXT, isPrimary
INTEGER)'); });
```

### 5. Verify the Database:

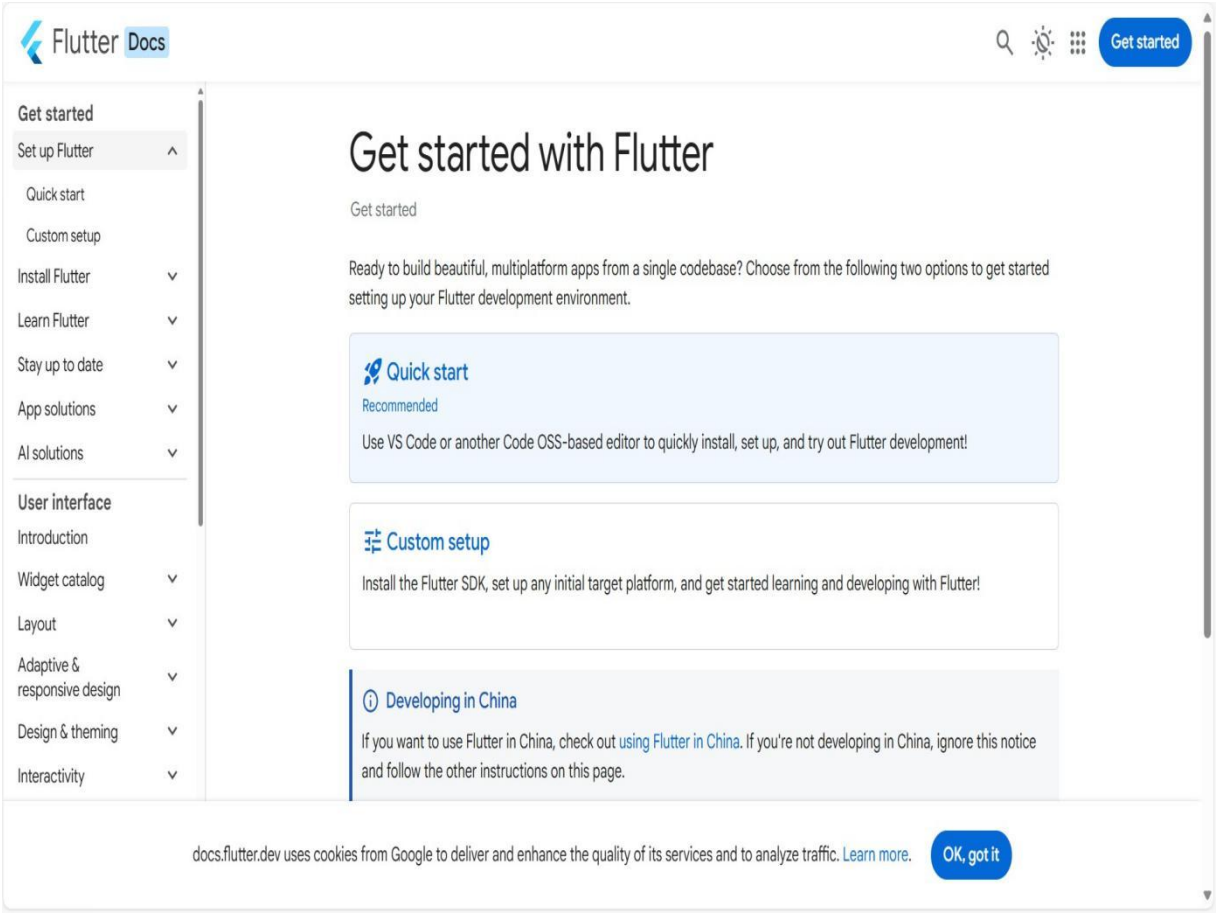
– Run the application on an emulator or device.
– Add an emergency contact and verify that it is stored and retrieved from SQLite.

# 4 Installation of Flutter (with PATH Setup)

## Download Flutter SDK

**Step 1:**

- Visit: https://docs.flutter.dev/get-started/install.
- Choose OS (Windows/Mac/Linux).
- Download Flutter SDK (Stable Channel).
- Extract it:
  - Windows: C:\src\flutter
  - Linux/Mac: $HOME/flutter




**Insatllation of flutter**

## Add Flutter to PATH

**Step 2:**

- Windows:
  1. Go to Control Panel → System → Advanced system settings.
  2. Environment Variables → Edit Path.

3. Add: C:\src\flutter\bin.

– Linux/Mac:

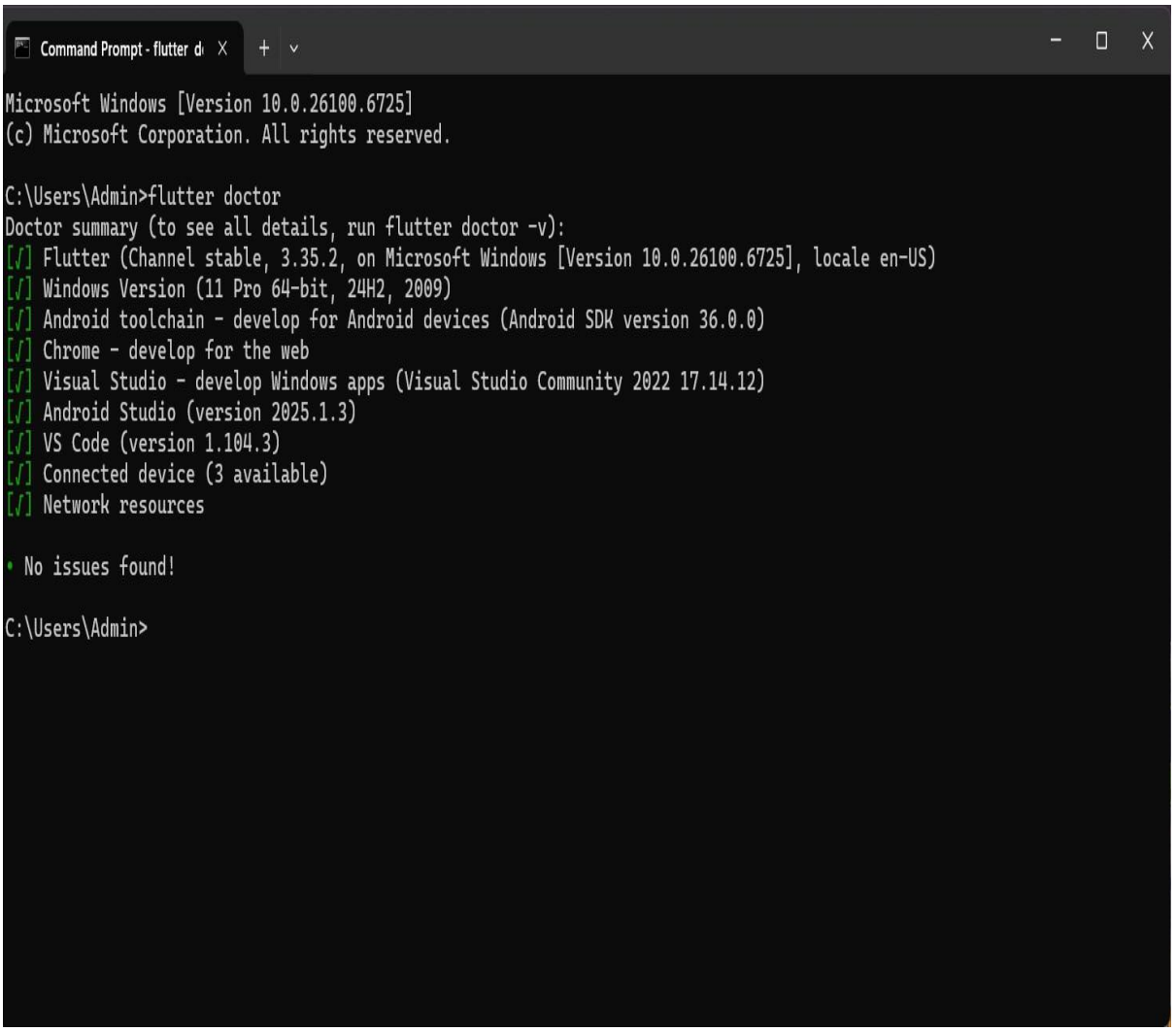
```
export PATH="$PATH:$HOME/flutter/bin"
```

Add this line to .bashrc or .zshrc.

## Verify Installation

**Step 3:**

```
flutter doctor
```




**flutter doctor** Command Showing Environment Setup

# Installation of Android Studio

## Step 1: Download and Install

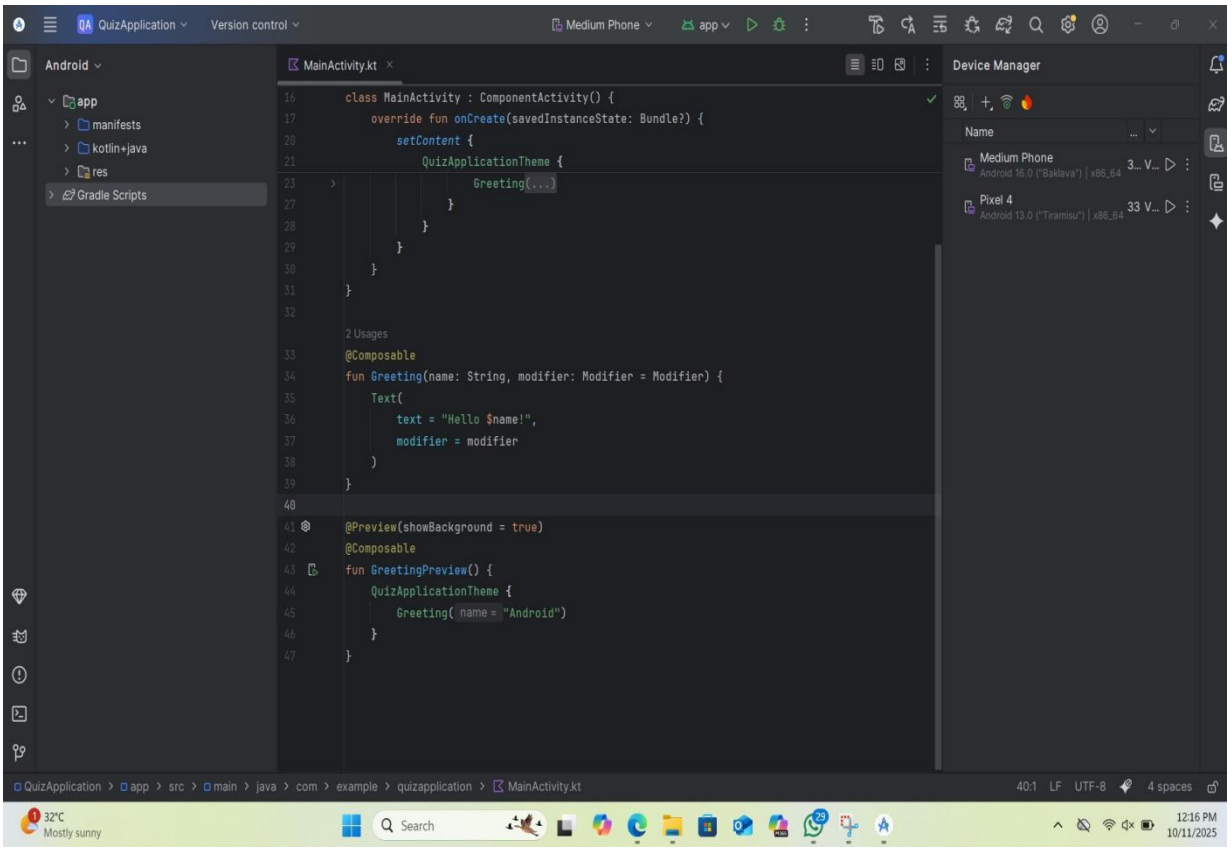
- Go to: https://developer.android.com/studio.
- Download Android Studio.
- Install with default settings.

## Step 2: Install Plugins

- Open Android Studio.
- Go to File → Settings → Plugins.
- Install Flutter (Dart installs automatically).

## Step 3: Configure SDK Path

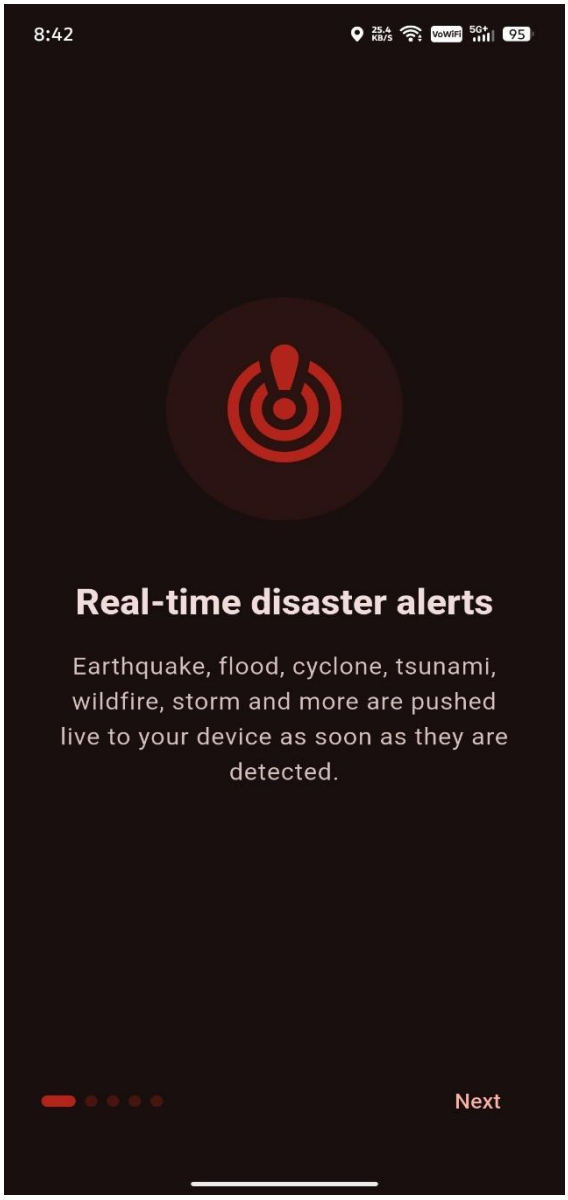
- Open SDK Manager.
- Verify Android SDK location.
- Install required SDK platforms & tools.




**Android Studio Interface Showing Project Workspace**

## Emulator Setup and Running

- Open Android Studio.
- Go to Tools → AVD Manager.
- Click Create Virtual Device.
- Choose a device model (e.g., Pixel 6).
- Select a system image (Android 13 or latest).
- Finish and launch emulator.




**the emulator image.**

# 5 Installation of Dart SDK in Windows

Dart is a programming language developed by Google, mainly used for building mobile, desktop, web, and server applications. The following steps describe how to install the Dart SDK on a Windows system.

1. **Download the Dart SDK:**
   - Go to the official Dart SDK website: https://dart.dev/get-dart.
   - Under the **Windows** section, choose the appropriate version (64-bit recommended) and download the ZIP file.
2. **Extract the Dart SDK:**
   - After downloading, extract the ZIP file to a preferred location, for example: C:\dart_sdk
3. **Set the Environment Variable:**
   - Right-click on **This PC** → **Properties** → **Advanced system settings**.
   - Click on **Environment Variables**.
   - Under **System variables**, select **Path** → **Edit**.
   - Click **New** and add the path to the bin folder inside Dart SDK, e.g., C:\dart_sdk\bin
4. **Verify the Installation:**
   - Open the **Command Prompt**.
   - Run the following command to verify that Dart is installed successfully:

     ```
     dart --version
     ```

5. **Output:** The terminal should display the installed Dart version, for example:

   ```
   Dart SDK version: 3.2.0 (stable)
   ```

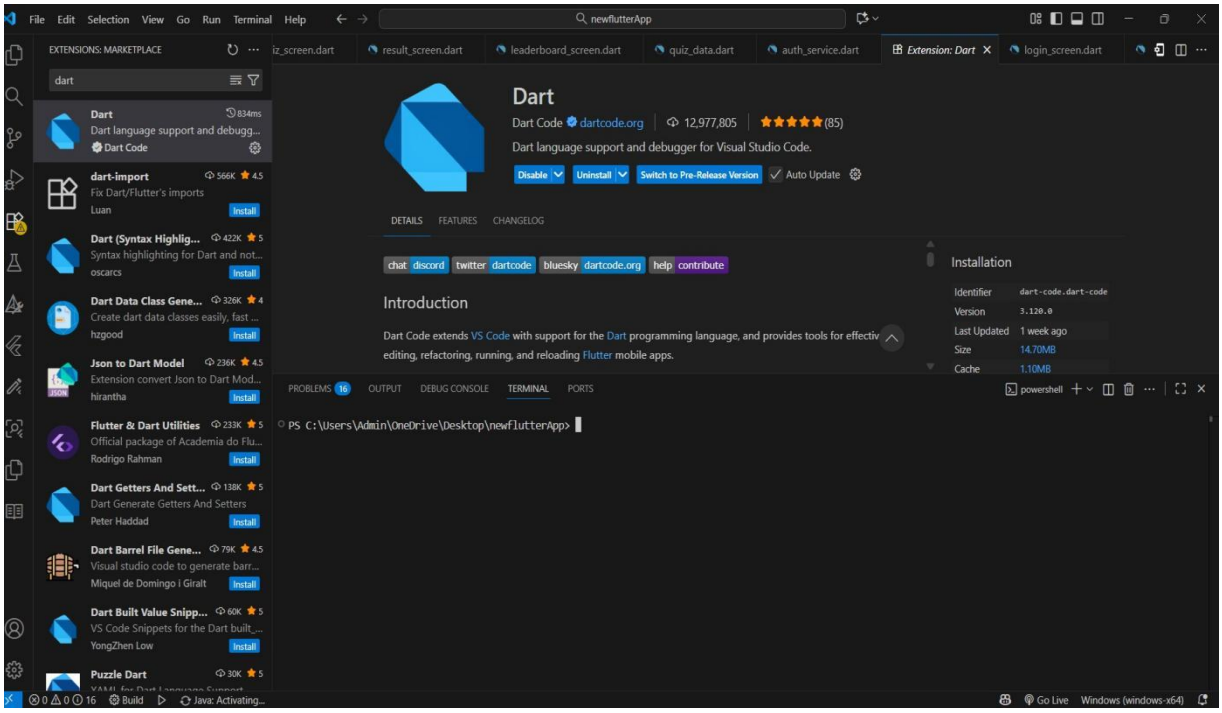
Once the Dart SDK is installed and configured, you can use it to compile and run Dart programs or build Flutter applications.

# 6 Installing Visual Studio Code

**Visual Studio Code (VS Code)** is a lightweight yet powerful open-source code editor developed by Microsoft. It is widely used for Flutter and Dart development because of its excellent support for extensions, debugging, IntelliSense, and inte-gration with the Flutter SDK. By installing VS Code, developers can write, test, and debug Flutter applications efficiently across multiple platforms.

## Steps to Install VS Code and Set Up for Flutter Development

1. **Download VS Code:** Visit the official Visual Studio Code website at https: //code.visualstudio.com/ and download the version suitable for your op-erating system (Windows, macOS, or Linux).
2. **Install VS Code:** Run the downloaded installer and complete the installation using the default settings. This ensures all necessary components, including PATH configuration, are properly set up.
3. **Install Extensions:** After launching VS Code, navigate to the Extensions panel using Ctrl + Shift + X. Search for and install the following essential extensions that are flutter,dart.
4. **Create a New Flutter Project: Press Ctrl + Shift + P to open the Command Palette and search for Flutter: New Project. Select a project type (e.g., Application), specify a project name (e.g., disasterlive_alerts), and choose a desired directory to create your new project.**
5. **Open the Project in VS Code:** Once the project is created, VS Code au-tomatically opens the folder. You can explore folders such as lib/, android/, and ios/ which are automatically generated by Flutter.




**Installing the Dart and Flutter extensions in Visual Studio Code**

# 7 Creating a New Flutter Project

Flutter is a framework developed by Google for building cross-platform applications using the Dart programming language. Before creating a new Flutter project, ensure that the Flutter SDK is installed and added to your system PATH.

## Steps to Create a New Flutter Project

1. **Open Command Prompt or Terminal.**
2. **Navigate to the desired folder:**

   cd C:\ Users\ <your - contactName >\ Documents

3. **Create a new Flutter project:**

   ```
   flutter create my_first_app
   ```

4. **Navigate into the project directory:**

   ```
   cd my_first_app
   ```

5. **Open the project in Visual Studio Code (or any preferred IDE):**

   **code .**

   Run the Flutter application:

   ```
   flutter run
   ```

**6.Verify successful installation:** Once the app runs, a default Flutter counter application appears on the emulator or connected device.

## Project Structure

After running the above commands, Flutter automatically generates the following project structure:

- lib/ – Contains the main Dart source code.
- android/, ios/, web/ – Platform-specific code.
- pubspec.yaml – Configuration file for dependencies and assets.
- test/ – Contains test files for unit and widget testing.
- assets/ – Directory to store images, fonts, and other static resources (added manually when need

# 8 Dart Basic Programming

Dart is a modern, object-oriented programming language developed by Google. It is optimized for building client applications, particularly mobile apps using the Flutter framework. Dart provides features like strong typing, asynchronous programming, and a rich standard library, making it both easy to learn and powerful for application development.

The following sections explain the fundamental programming concepts in Dart.

## Variables

Variables are containers that hold data values. In Dart, every variable has a **name**, a **type**, and a **value**. Dart is a statically typed language, which means the data type of a variable is known at compile-time. However, Dart also supports type inference using the var keyword.

- **String:** Stores textual data.
- **int:** Stores whole numbers.
- **double:** Stores decimal numbers.
- **bool:** Stores Boolean values (true or false).
- **var:** Automatically infers the data type from the assigned
- **const:** Declares a compile-time constant — the value must be known and fixed at compile time.
- **final:** Declares a variable whose value can be set only once, but is determined at runtime.
- Dart is **type-safe**, meaning you cannot assign a value of a different type once declared.

**Code:**

```
void main() {
  String name = '
  Lalithya'; int age
  = 22;
  double height =
  5.6; bool
  isStudent = true;
  print('Name:
  $name'); print('
  Age: $age');
  $height'); print('
  Student: $isStudent');}
```

**Output:**

```
Name:Dinesh
Age: 22
Height: 5.6
Student: true
```

## Conditional Statements

Conditional statements are used to make decisions in a program. They allow dif-ferent actions to be executed based on whether certain conditions are true or false. In Dart, conditional logic is implemented using **if,** else **if,** and else statements.

- **if:** Executes a block if the condition is true.
- **else if:** Checks another condition if the previous one is false.

**Code:**

```
void main(){
  int marks = 85;
  if (marks >=
    90) {
    print('
    Grade:A+');
  } else if (marks >= 75){
    print('Grade: A');
  }   else    {
    print('
    GradeB');
  }
  }
```

**Output:**

```
Grade: A
```

## Loops

Loops are used to repeatedly execute a block of code as long as a specified condition is true. They help automate repetitive tasks and reduce redundancy. Dart supports the following types of loops:

- **for loop:** Repeats code a fixed number of times.
- **while loop:** Repeats while a condition is true.
- **do-while loop:** Executes code once, then checks the condition.

**Code:**

```
void main() {
  for (int i = 1; i <= 3;
    i++) {print('
    Iteration: $i');
  }
}
```

**Output:**

```
Iteration: 1
Iteration: 2
Iteration: 3
```

## Lists

A List in Dart is an ordered collection of elements. Lists are similar to arrays in other programming languages. They can store multiple values of the same or different data types and allow random access using indexes.

**Code:**

```
void main() {
  List<String>fruits = ['Apple', 'Banana', '
  Mango']; fruits.add('Orange');
  print('Fruits List: $fruits');}
```

**Output:**

```
Fruits List: [Apple, Banana, Mango, Orange]
```

## Maps

Maps in Dart are used to store data as key-value pairs, where each key is unique. They are similar to dictionaries in Python or hash maps in Java. Maps allow fast data retrieval using keys.

**Code:**

```
void main() {
  Map<String, int>
    scores = {'Alice
    ': 90,
    'Bob': 80,
    'Charlie ':
  95}; scores.forEach((
  name, mark) {
    print('$name scored $mark marks');
});}
```

**Output:**

```
Alice scored 90 marks Bob scored
80 marks Charlie scored 95 marks
```

## Functions

Functions are reusable blocks of code designed to perform specific tasks. They help make programs modular and easier to maintain. In Dart, functions can take parameters and return values.

**Code:**

```
int addNumbers(int a,
  int b) {return a + b;}
void main() {
int result = addNumbers(5, 7); print('Sum: $result');}
```

**Output:**

```
Sum: 12
```

# 9 Case Study on Disaster Alert App

The Disaster Alert App brings together a critical-status dashboard, emergency helplines, hazard categories, staged safety guidance, weather, live alerts, a map, alert details, SOS, emergency contacts, and safety settings.

## Key Features

- **Dashboard**: Critical status, alert counts, GPS status, and emergency helplines.
- **Survival Guide:** Earthquake, flood, cyclone, tsunami, landslide, wildfire, and heavy-rain categories.
- **Hazard Guidance:** Safety steps grouped into Before, During, and After.
- **Weather**: Current conditions, humidity, wind, rainfall, hourly data, and a three-day forecast.
- **Live Map:** User location, hazard filters, markers, and alert details.
- **Live Alerts:** Hazard feed with category/status filters, source, distance, and safety labels.
- **SOS and Contacts**: SOS action and contact form with name, phone, relationship, and primary setting.
- **Safety Settings**: Location, alert radius, and disaster-type filters.

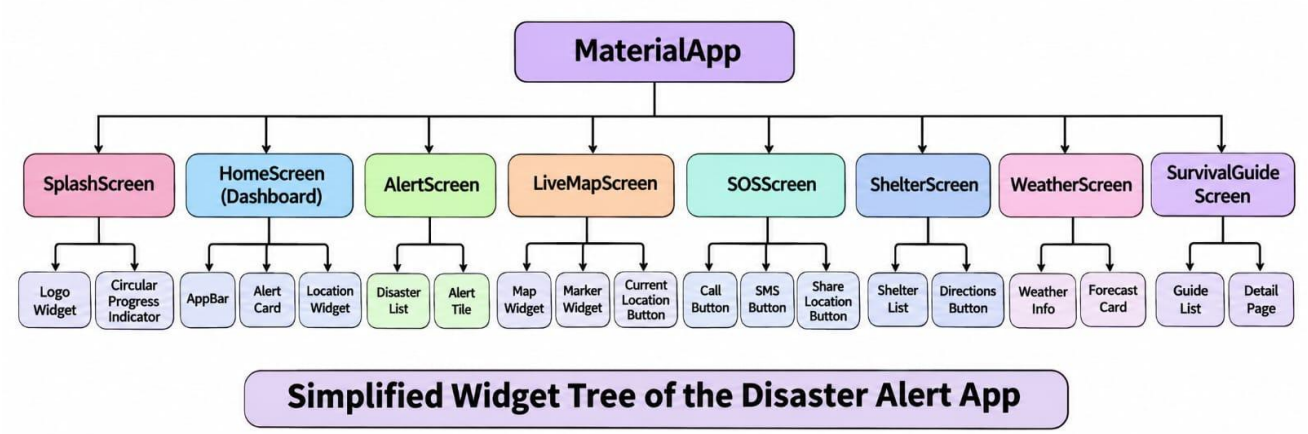
## Technologies Used

- **Flutter SDK:** For building cross-platform UI and managing widgets.
- **Dart Language:** The core programming language for Flutter develop-ment.
- **VS Code:** Integrated development environment used for coding and de-bugging.
- **Android Emulator:** Used for testing and visualizing the app during development.
- **REST API:** For fetching and updating leaderboard data dynamically.
- **Provider Package:** For state management and real-time UI updates.
- **GitHub:** For version control and project collaboration.

# 10 Exploring Flutter Widgets

**Objective: Explore Text, Icon, Container, Card, ListTile, Column, Row, and ListView.**
**Relation to Project: These widgets appear in the dashboard, helplines, hazard cards, weather panels, alert cards, and map detail sheet.**
**Description: The dashboard uses text, icons, containers, and cards to show key information. Hazard categories use selectable cards, while the live -alert feed uses repeated alert cards and filters.**
Reusable widgets can also display weather summaries and contact rows.
Cards group related information and separate alert types.
Icons help identify hazards and emergency actions.
Scrollable lists keep longer guidance and alerts accessible.

**Widgets Flow**



Simplified Widget Tree of the Disaster Alert App

**Widget Structure:** Scaffold provides the screen; Cards and Containers group information; Text and Icon display content; ListView and Column arrange repeated items.

- The dashboard banner shows the current alert state.
- Summary cards show active alerts, nearby alerts, and GPS status.
- Helpline rows pair service labels with icons and call actions.
- Hazard and alert cards use clear labels and contrasting colours.

**Hazard Card code:**

```
import 'package:flutter/material.dart';
HazardCard extends StatelessWidget {
final String title;
final IconData icon;

const HazardCard({
    super.key,
required this.title,
required this.icon,
  });
  @override
  Widget build(BuildContext context) {
return Card(
      child: ListTile(
```

```
            leading: const Icon(Icons.person),
            title: Text(
              name,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
      subtitle: Text(category),
      trailing: ElevatedButton(
              onPressed: () {},
              child: const Text('Request'),
            ),
          ),
        );
      }
    }
```

## Output:




**Survival Guide Screen**

**Description:**

The AlertCard widget is a custom Flutter component used to display an individual emergency contact's details in the Disaster Alert App. It shows the emergency contact's name, service category, and availability in a visually structured card format..

**Functionality:**

Each emergency contact entry is represented using a Card and ListTile widget. The emergency contact's contact icon is displayed on the left, the emergency contact's name appears as the title, and the service category is shown below the name. A Request button is provided on the right side of the tile to allow users to initiate a SOS action.

**Constructor Parameters:**

- name: The name of the emergency contact.
- category: The service category offered by the emergency contact.

**Widget Structure:**

The widget uses the `ListTile` layout with the following components:

- **leading:** An `Icon` widget displaying a person icon.
- **title:** A bold Text widget displaying the emergency contact's name.
- **subtitle:** A Text widget displaying the emergency contact's service category.
- **trailing:** An ElevatedButton widget labeled Open to view a emergency contact SOS action.

**Live AlertsTile code:**

```
import 'package:flutter/material.dart';
import '../services/emergency contact_service.dart';
import 'survival_guide_screen.dart';
class FindEmergency ContactsScreen extends StatefulWidget {
  const FindEmergency ContactsScreen({super.key});
  @override
  State<FindEmergency ContactsScreen> createState() =>
      _FindEmergency ContactsScreenState();
}
class _FindEmergency ContactsScreenState
    extends State<FindEmergency ContactsScreen> {
  late Future<List<Map<String, dynamic>>> _emergency contactsFuture;
  String searchQuery = "";
  String categoryFilter = "All";
  @override
  void initState() {
    super.initState();
    _emergency contactsFuture = Emergency ContactService.getAll();
  }
  IconData _availabilityIcon(bool available) {
    return available ? Icons.check_circle : Icons.schedule;
  }
```

```
          Color _availabilityColor(bool available) {
          return available ? Colors.green : Colors.orange;
            }
          Widget _buildAnimatedEntry({
              required int index,
              required Widget child,
            }) {
              return TweenAnimationBuilder<double>(
                tween: Tween(begin: 0, end: 1),
                duration: Duration(milliseconds: 400 + (index * 100)),
                curve: Curves.easeOut,
                builder: (context, value, _) {
                  return Opacity(
                    opacity: value,
                    child: Transform.translate(
                      offset: Offset(0, 20 * (1 - value)),
                      child: child,
                    ),);},);}
            List<Map<String, dynamic>> _filterEmergency Contacts(
              List<Map<String, dynamic>> emergency contacts,
            ) {
              return emergency contacts.where((emergency contact) {
                final name =
                    (emergency contact['name'] ??
      '').toString().toLowerCase();
                final category =
                    (emergency contact['category'] ?? '').toString();
                final matchesSearch =
                    name.contains(searchQuery.toLowerCase());
                final matchesCategory = categoryFilter == 'All' ||
                    category == categoryFilter;
                return matchesSearch && matchesCategory;
              }).toList();
            }
            @override
            Widget build(BuildContext context) {
              return Scaffold(
                appBar: AppBar(
                  title: const Text('Find Emergency Contacts'),
                  actions: [
                    IconButton(
                      icon: const Icon(Icons.home),
                      tooltip: 'Go to Survival Guide',
                      onPressed: () {
                        Navigator.pushReplacement(
                          context,
                          MaterialPageRoute(
                            builder: (_) => const SurvivalGuideScreen(),
                          ),);},),],),
                body: Container(
                  decoration: const BoxDecoration(
                    gradient: LinearGradient(
```

```
            colors: [
                        Colors.purpleAccent,
                        Colors.deepPurple,
                        Colors.pinkAccent,
             ],
                      begin: Alignment.topLeft,
                      end: Alignment.bottomRight,
                    ),
                  ),
                  child: Column(
                    children: [
                      Padding(
                        padding: const EdgeInsets.all(12),
                        child: TextField(
                          onChanged: (value) {
                            setState(() => searchQuery = value);
                          },
                          decoration: InputDecoration(
                            hintText: 'Search emergency contacts...',
                            prefixIcon: const Icon(Icons.search),
                            filled: true,
                            fillColor: Colors.white,
                            border: OutlineInputBorder(
                              borderRadius: BorderRadius.circular(12),
                            ),
                          ),
                        ),
                      ),
                      Padding(
                        padding: const EdgeInsets.symmetric(
                          horizontal: 12,
                        ),
                        child: DropdownButtonFormField<String>(
                          value: categoryFilter,
                          decoration: const InputDecoration(
                            filled: true,
                            fillColor: Colors.white,
                            labelText: 'Service Category',
                            border: OutlineInputBorder(),
                          ),
                          items: [
                            'All',
                            'Education',
                            'Health',
                            'Environment',
                          ].map((category) {
                            return DropdownMenuItem(
                              value: category,
                              child: Text(category),
                            );}).toList(),
```

```
                                    onChanged: (value) {
                            setState(() {
                              categoryFilter = value ?? 'All';
                            });}, ),),
        Expanded(
                        child: FutureBuilder<
                            List<Map<String, dynamic>>>(
                          future: _emergency contactsFuture,
                          builder: (context, snapshot) {
                            if (snapshot.connectionState ==
                                ConnectionState.waiting) {
                              return const Center(
                                child: CircularProgressIndicator(
                                  color: Colors.white,
                                ),
                              );
                            }

                            if (snapshot.hasError) {
                              return const Center(
                                child: Text(
                                  'Unable to load emergency contacts.',
                                  style: TextStyle(color: Colors.white),
                                ),
                              );
                            }

                            final emergency contacts =
                                _filterEmergency Contacts(snapshot.data ?? []);

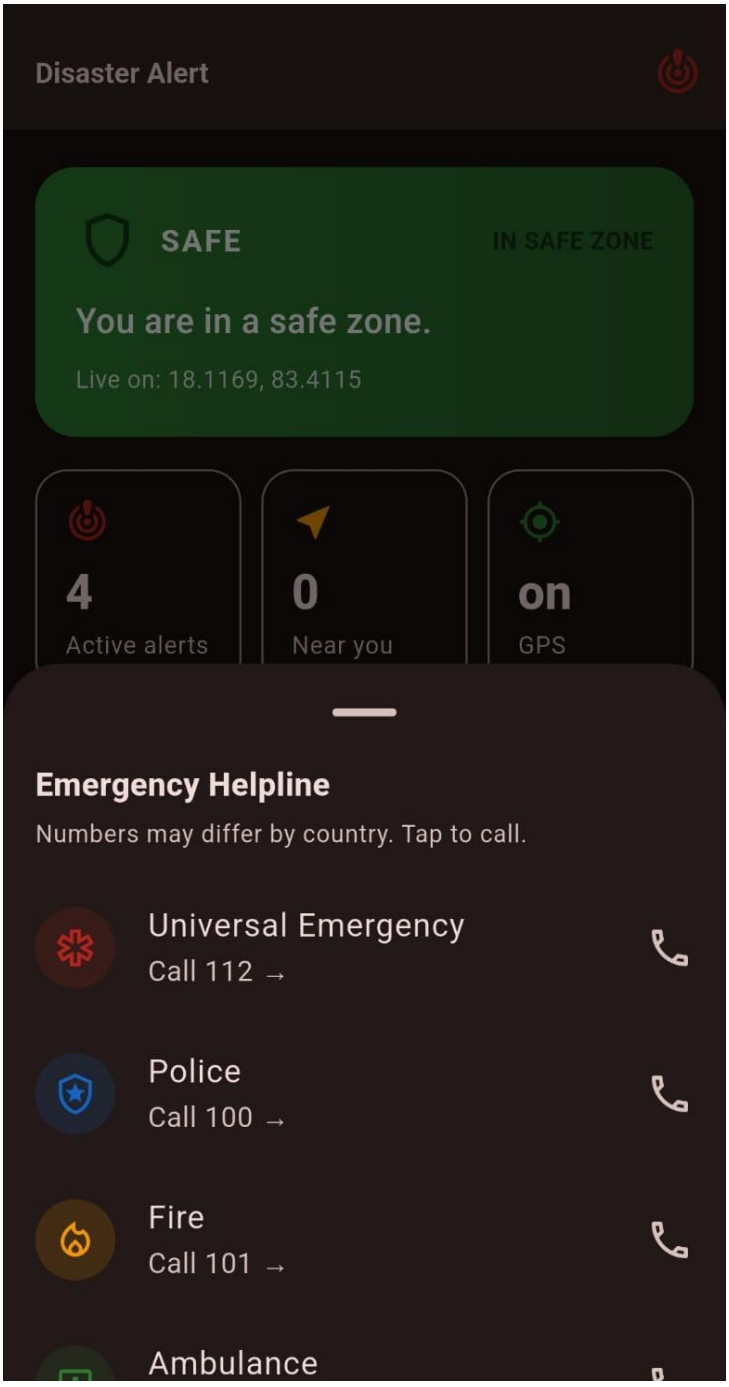
                            if (emergency contacts.isEmpty) {
                              return const Center(
                                child: Text(
                                  'No emergency contacts found.',
                                  style: TextStyle(color: Colors.white),
                                ),
                              );
                            }

                            return ListView.builder(
                              itemCount: emergency contacts.length,
                              itemBuilder: (context, index) {
                                final emergency contact = emergency
contacts[index];
                                final available =
                                    emergency contact['available'] == true;

                                return _buildAnimatedEntry(
                                  index: index,
                                  child: Card(
```

```
                        child: Icon(Icons.person),
                                    margin: const EdgeInsets.symmetric(
                                      horizontal: 12,
                                      vertical: 6,
                                    ),
                                    child: ListTile(
                                      leading: const CircleAvatar(
                         title: Text(
                                   emergency contact['name']?.toString()??
                           'Emergency Contact',
                           style: const TextStyle(
                          fontWeight: FontWeight.bold,
                           ),
                         ),
                             subtitle: Text(
                              '${emergency contact['category'] ??
    'General'}\n'
                               '${emergency contact['description'] ?? ''}',
                                      ),
                             isThreeLine: true,
                              trailing: Icon(
                             _availabilityIcon(available),
                              color: _availabilityColor(available),
                        );},),),],),),);}}
```

**Output:**




**Emergency Helpline Screen**

# 11 Responsive UI Design

**Objective: Design a responsive layout using Row, Column, and MediaQuery. Relation to Project: The survivalGuide adapts to screen sizes across devices.**

**Description :**

The SurvivalGuideScreen in the Disaster Alert App implements responsive UI design principles in Flutter, ensuring that the layout adapts to different screen sizes and device orientations. Widgets such as SafeArea and SingleChildScrollView help prevent content overflow and keep important information visible on devices with notches or smaller displays.The interface uses flexible layout widgets such as Row, Column, Expanded, and Container to organize emergency contact details, service categories, and SOS action options. The MediaQuery widget can be used to adjust spacing, padding, and card layouts according to the available screen width and height.The emergency contact cards and service category sections can reposition themselves based on the available screen space. Scrolling allows users to access all emergency contact details and SOS action options on smaller screens.Animations and smooth transitions can further enhance user engagement while maintaining a responsive interface. Overall, the Disaster Alert App aims to provide a visually consistent, user-friendly, and adaptable experience across different devices.

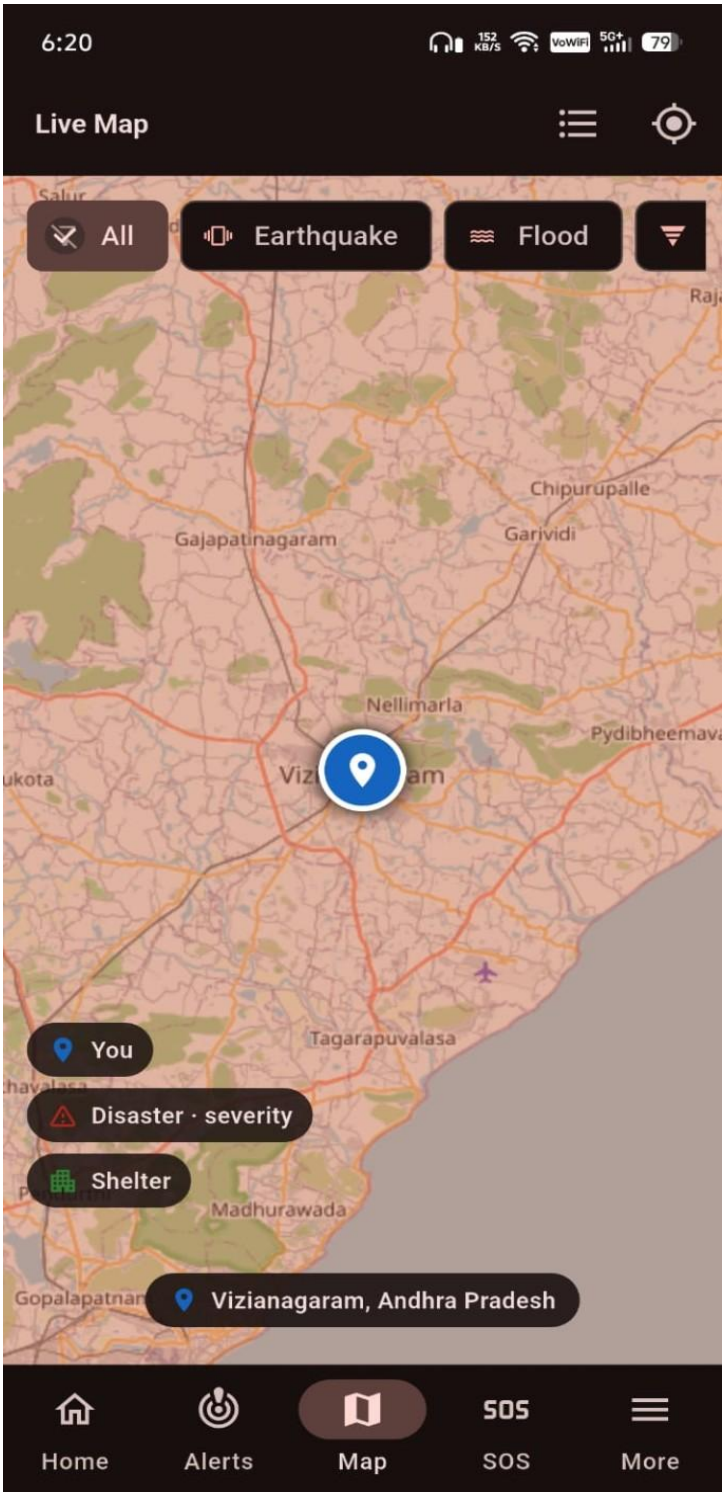
**Survival Guide Screen code:**

```
class SurvivalGuideScreen extends StatefulWidget {
  const SurvivalGuideScreen({super.key});
  @override
  State<SurvivalGuideScreen> createState() =>
      _SurvivalGuideScreenState();
}
class _SurvivalGuideScreenState extends
State<SurvivalGuideScreen> {
  @override
  Widget build(BuildContext context) {
    final width = MediaQuery.of(context).size.width;
    return Scaffold(
      appBar: AppBar(
        title: const Text('Disaster Alert'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(16),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              const CircleAvatar(
                radius: 40,
                backgroundColor: Colors.deepPurple,
                child: Icon(
                  Icons.sos,
                  size: 45,
                  color: Colors.white,
                ),
              ),
```

```
const SizedBox(height: 16),
                          const Text(
                            'Welcome to Disaster Alert!',
                            style: TextStyle(
                              fontSize: 22,

              fontWeight: FontWeight.bold,
                            ),
                          ),
                          const SizedBox(height: 20),
                          Wrap(
                            spacing: 12,
                            runSpacing: 12,
                            children: [
                              _buildCard(
                                width,
                                Icons.search,
                                'Find Emergency Contacts',
                                Colors.deepPurple,
                              ),
                              _buildCard(
                                width,
                                Icons.assignment,
                                'My Requests',
                                Colors.teal,
                              ),
                              _buildCard(
                                width,
                                Icons.person,
                                'Emergency Contacts',
                                Colors.orange,
                              ),
                            ],
                          ),
                        ],
                      ),
                    ),
                  ),
                );
              }
                Widget _buildCard(
                double screenWidth,
                IconData icon,
                String title,
                Color color,
              ) {
                return SizedBox(
                  width: screenWidth > 600
                      ? (screenWidth - 56) / 2
                      : screenWidth - 32,
                  child: Card(
                    elevation: 4,
                    child: ListTile(
                      leading: Icon(icon, color: color, size: 32),
                      title: Text(
                        title,
                        style: const TextStyle(
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      trailing: const Icon(Icons.arrow_forward_ios),
                    ),),);},}
```

**Output:**




**Survival Guide Screen**

# 12 Navigation Between Screens

**Objective:** To implement navigation using Navigator and named routes. Relation to Project:Used to move between the SOS, Add Contact, Survival Guide, Find Emergency Contacts, Emergency Contact Details, and My Requests screens.

**Explanation:**

- Users start at the SosScreen.
- After successful contact management, users are navigated to the SurvivalGuideScreen.
- From the SurvivalGuideScreen, users can:
- Access FindEmergency ContactsScreen to search for emergency contacts.
- Access Emergency ContactDetailsScreen to view emergency contact information.
- Access MyRequestsScreen to track SOS actions.
- Access NewContactScreen to create a new account
- Logging out from the Survival Guide:
- Redirects the user back to the SosScreen.
- Clears the navigation stack to prevent returning to saved screens using the back button.

**Sample Code:**

```
import 'package:flutter/material.dart';
void main() {
  runApp(MaterialApp(
    initialRoute: '/',
    routes: {
      '/': (context) => const SosScreen(),
      '/survivalGuide': (context) => const SurvivalGuideScreen(),
      '/findEmergency Contacts': (context) =>
          const FindEmergency ContactsScreen(),
      '/myRequests': (context) => const MyRequestsScreen(),
    },
  ));
}
Navigator.pushNamed(context, '/findEmergency Contacts');
Navigator.pushNamed(context, '/myRequests');
Navigator.pop(context);
Navigator.pushNamedAndRemoveUntil(
  context,
  '/',
  (route) => false,
);
```

# 13 Stateful and Stateless Widgets

**Objective:** Understand widget lifecycles and state management.

**Relation to Project:Used in the Disaster Alert App to display emergency contact details using stateless widgets and manage dynamic data using stateful widgets.**

**Description:**In Flutter, widgets are the building blocks of the user interface and are classified as either Stateless or Stateful, depending on whether they maintain internal state that can change during the widget's lifecycle

.∗ **Stateless Widgets:**

Stateless widgets do not maintain mutable internal state and are immutable once created. They rebuild when their parent widget provides updated configuration.

In the Disaster Alert App, examples include:

- **AlertCard –** Displays a emergency contact's name and service category using input parameters.
- **CustomButton** – Displays a button with a predefined label and style, with actions
- handled through callbacks.

Stateless widgets are used when the UI content depends only on the input parameters and does not change dynamically within the widget itself.

**Syntax for StatelessWidget:**

```
class AlertCard extends StatelessWidget {
            final String name;
          final String category;

            const AlertCard({
                 super.key,
            required this.name,
          required this.category,
                    });

                @override
    Widget build(BuildContext context) {
              return Container(
                child: Column(
                  children: [
                   Text(name),
                  Text(category),
                        ],
                      ),
                    );
                   }
                  }
```

* **Stateful Widgets:**

- Stateful widgets maintain mutable state that can change over time. They can rebuild their interface whenever their internal state changes. In the Disaster Alert App, examples include:
- **SurvivalGuideScreen –Displays the logged-in user's information and updates when contact details change.**
- **FindEmergency ContactsScreen – Tracks search text and selected service category to filter emergency contacts dynamically**

Stateful widgets are essential for interactive UI components that respond to user input or asynchronous data changes.

**Syntax for Stateful Widget:**

```
class FindEmergency ContactsScreen extends StatefulWidget {
  const FindEmergency ContactsScreen({super.key});

  @override
  State<FindEmergency ContactsScreen> createState() =>
      _FindEmergency ContactsScreenState();
}

class _FindEmergency ContactsScreenState
    extends State<FindEmergency ContactsScreen> {
  @override
  Widget build(BuildContext context) {
    return Container(
      // Disaster Alert UI code here
    );
  }
}
```

# 14 Custom Widgets and Styling

**Objective:** Create custom reusable widgets.

**Description:**

The Disaster Alert App uses custom widgets and consistent styling to improve modularity, maintainability, and visual appearance. Custom widgets encapsulate UI components that can be reused across multiple screens, reducing code duplication and maintaining a uniform design

* **Custom Widgets:**

- AlertCard: A reusable card widget that displays a disaster ques-tion along with multiple options. Each option is a styled button that triggers a callback when selected.
- RequestTile: A reusable list tile to display user rank, name, and severity with a circular avatar and trailing severity.
- CustomButton: A reusable button component with configurable text, icon, color, and onPressed action. Used in emergency-contact management, SOS, and navigation buttons.

* **Styling Approach:**

- Consistent color scheme using shades of deepPurple and complemen-tary accent colors for gradients (e.g., option cards and survivalGuide back-grounds).
- Rounded corners on cards, buttons, and text fields for a modern look.
- Gradients and shadows applied to navigation cards on the Survival Guide for depth and visual hierarchy.
- Bold and large text styles for headings (contactName, hazard title, titles) to improve readability.
- Padding, margins, and spacing are used consistently to maintain visual balance and responsive layout across different screen sizes.

- Icons combined with text in buttons and list tiles enhance usability and quick recognition of actions.

∗ **Benefits:**

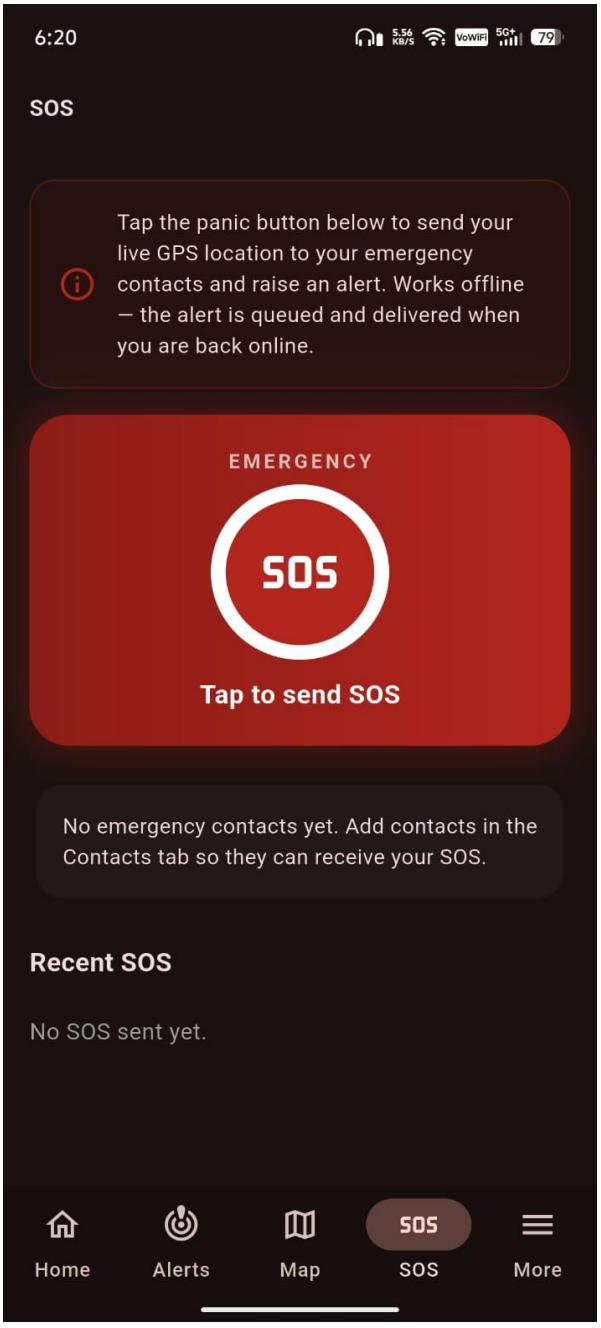
- Enhances code reusability by creating modular components.
- Maintains a consistent look and feel throughout the application.
- Simplifies maintenance and scalability when adding new screens or UI elements.
- Improves readability, accessibility, and user engagement through well-thought-out styling.

**Relation to Project: Used for buttons and live_alerts tiles.**

**Custom Button code:**

```
import 'package:flutter/material.dart';
class CustomButton extends StatelessWidget {
  final String text;
  final VoidCallback onPressed;
  final Color color;
  final IconData? icon;
  const CustomButton({
    super.key,
    required this.text,
    required this.onPressed,
    this.color = Colors.deepPurple,
    this.icon,
  });
  @override
  Widget build(BuildContext context) {
    return ElevatedButton.icon(
      style: ElevatedButton.styleFrom(
        backgroundColor: color,
        padding: const EdgeInsets.symmetric(
          horizontal: 20,
          vertical: 14,
        ),
      ),
      icon: icon != null
          ? Icon(icon, size: 20, color: Colors.white)
          : const SizedBox.shrink(),
      label: Text(
        text,
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          color: Colors.white,
        ),
      ),
      onPressed: onPressed,
    );
   }
```

**Output :**




**SOS Screen**

# 15 Forms and Validation

**Objective:** Create a form with validation.

**Relation to Project: Used in New Contact and emergency-contact workflows.**
**to Project: Used in SOS and New Contact screens.**

**Description:**

The NewContactScreen implements a user contact saving form using multiple TextField widgets to collect input for contactName, phone, phone number, relationship, and confirm relationship. It demonstrates basic form validation and user feedback mechanisms:

* **Input Validation:**
  - Checks that all fields are filled before submission.
  - Ensures that the relationship and confirm relationship fields match.
  - Displays error messages dynamically if validation fails.
* **User Feedback:**
  - Shows a CircularProgressIndicator while contact saving is in progress.
  - Displays success messages via SnackBar upon successful contact saving.
* **Navigation:**
  - On successful contact saving, the user is navigated to the SurvivalGuideScreen using Navigator.pushAndRemoveUntil() to clear the navigation stack.
  - Provides a “Back to SOS” button to navigate back to the SosScreen.
* **Code Structure:**
  - Uses TextEditingController for each input field to read and manage user input.
  - The saveContact() method encapsulates validation logic, loading state management, and calls to DatabaseHelper.saveContact() for actual regis-tration.
  - Error messages and loading indicators are updated dynamically using setState(),demonstrating a typical use **stateful widgets** in form.

**New Contact Screen code:**

```
import 'package:flutter/material.dart';
import '../services/database_helper.dart';
import '../widgets/custom_button.dart';
import 'survival_guide_screen.dart';
import 'sos_screen.dart';

class NewContactScreen extends StatefulWidget {
  const NewContactScreen({super.key});

  @override
  State<NewContactScreen> createState() => _NewContactScreenState();
}

class _NewContactScreenState extends State<NewContactScreen> {
  final _userCtrl = TextEditingController();
  final _phoneCtrl = TextEditingController();
  final _phoneCtrl = TextEditingController();
  final _passCtrl = TextEditingController();
  final _confirmPassCtrl = TextEditingController();
  bool _loading = false;
  String? _error;
  @override
  void dispose() {
    _userCtrl.dispose();
    _phoneCtrl.dispose();
    _phoneCtrl.dispose();
    _passCtrl.dispose();
    _confirmPassCtrl.dispose();
    super.dispose();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Add Emergency Contact'),
        backgroundColor: Colors.deepPurple,
        foregroundColor: Colors.white,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Icon(
                Icons.sos,
                size: 70,
                color: Colors.deepPurple,
              ),
              const SizedBox(height: 16),
              const Text(
                'Add Emergency Contact',
                textAlign: TextAlign.center,
```

```
                  style: TextStyle(
                    fontSize: 24,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 24),
                TextField(
                  controller: _userCtrl,
                  decoration: InputDecoration(
                    labelText: 'Contact Name',
                    prefixIcon: const Icon(Icons.person),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: _phoneCtrl,
                  keyboardType: TextInputType.phoneAddress,
                  decoration: InputDecoration(
                    labelText: 'Phone Number',
                    prefixIcon: const Icon(Icons.phone),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: _phoneCtrl,
                  keyboardType: TextInputType.phone,
                  decoration: InputDecoration(
                    labelText: 'Phone Number',
                    prefixIcon: const Icon(Icons.phone),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 16),
                TextField(
                  controller: _passCtrl,
                  obscureText: true,
                  decoration: InputDecoration(
                    labelText: 'Relationship',
                    prefixIcon: const Icon(Icons.lock),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 16),
                TextField(
```

```
                controller: _confirmPassCtrl,
                obscureText: true,
                decoration: InputDecoration(
                  labelText: 'Confirm Relationship',
                  prefixIcon: const Icon(Icons.lock_outline),
                  border: OutlineInputBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
              ),
              const SizedBox(height: 16),

              if (_error != null)
                Text(
                  _error!,
                  style: const TextStyle(color: Colors.red),
                ),

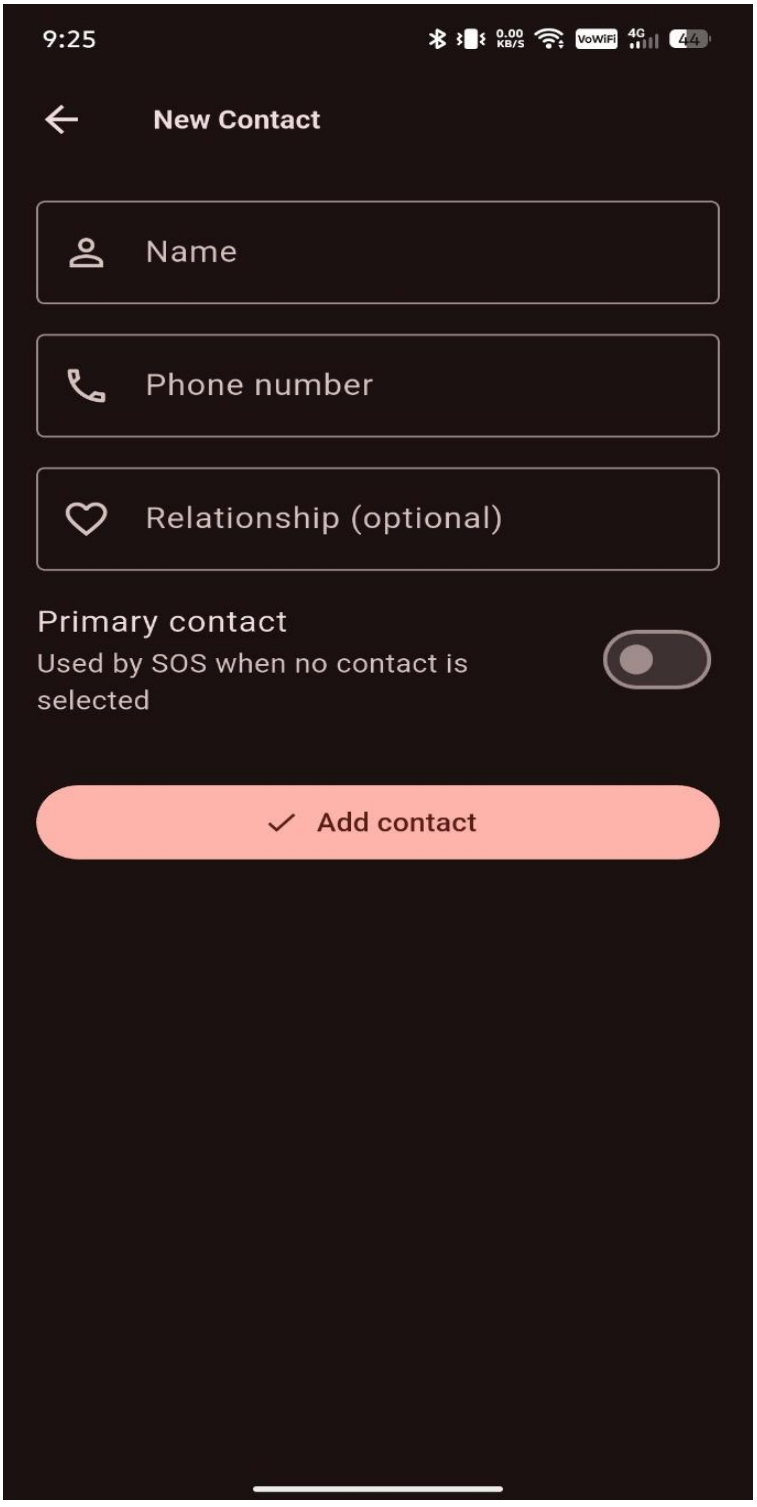
              CustomButton(
                text: _loading ? 'Saving...' : 'Add Contact',
                icon: Icons.person_add,
                onPressed: () {
                  setState(() {
                    _error = _passCtrl.text != _confirmPassCtrl.text
                        ? 'Relationships do not match'
                        : null;
                  });

                  if (_error != null) return;

                  // Save the contact using DatabaseHelper.
insertContact().
                },
              ),
              const SizedBox(height: 12),

              TextButton(
                onPressed: () {
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(
                      builder: (_) => const SosScreen(),
                    ),
                  );},
                child: const Text(
                  'Save contact and return',
                ),
              ),
            ],
          ),
        ),
      ),
    );
```

**Output:**




**New Emergency Contact Screen**

**Description:**

The SosScreen implements emergency contact management form using TextField wid-gets for contactName,relationship input.

It demonstrates form validation,error handling,and navigation after successful sendSOS:

* **Input Validation:**
  - Checks that both contactName, relationship fields are not empty before sub-mission.
  - Displays an error message dynamically if validation fails or authenti-cation fails.
* **User Feedback:**
  - Shows a CircularProgressIndicator while the sendSOS request is being processed.
  - Displays error messages below the input fields when sendSOS fails.
* **Navigation:**
  - On successful sendSOS, the user is navigated to the SurvivalGuideScreen us-ing Navigator.pushAndRemoveUntil() to clear the navigation stack and prevent back navigation to the sendSOS screen.
  - Provides a “Create an Account” button to navigate to the NewContactScreen to add an emergency contact.
* **Code Structure:**
  - Uses TextEditingController to manage the contactName and relationship input.
  - The sendSOS() method handles input validation, loading state man-agement, and calls DatabaseHelper.getContacts() for contact management.
  - Error messages and loading indicators are updated using setState(), demonstrating a typical use of a **stateful widget** for form handling.

**New Contact Screen code:**

```
import 'package:flutter/material.dart';
import '../services/database_helper.dart';
import '../widgets/custom_button.dart';
import 'new_contact_screen.dart';
import 'survival_guide_screen.dart';
class SosScreen extends StatefulWidget {
  const SosScreen({super.key});
  @override
  State<SosScreen> createState() => _SosScreenState();
}
class _SosScreenState extends State<SosScreen> {
  final _userCtrl = TextEditingController();
  final _passCtrl = TextEditingController();
  bool _loading = false;
  String? _error;
  @override
  void dispose() {
    _userCtrl.dispose();
    _passCtrl.dispose();
    super.dispose();
  }
  void _sendSOS() async {
    if (_userCtrl.text.trim().isEmpty ||
        _passCtrl.text.isEmpty) {
      setState(() {
        _error = 'Please enter contactName and relationship';
      });
      return;}
    setState(() {
      _loading = true;
      _error = null;
    });
    try {
      // Connect SQLite contact management through DatabaseHelper
here
      // This sample navigates to the Survival Guide after input
validation.
      if (!mounted) return;
      Navigator.pushAndRemoveUntil(
        context,
        MaterialPageRoute(
          builder: (_) => const SurvivalGuideScreen(),
        ),
        (route) => false,
      );
    } catch (e) {
      setState(() {
        _error = 'SOS failed. Please try again.';
      });
    } finally {
      if (mounted) {
        setState(() {
          _loading = false;
        });
      }
    }
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
```

```
                title: const Text('Emergency SOS'),
                backgroundColor: Colors.deepPurple,
                foregroundColor: Colors.white,
              ),
              body: SafeArea(
                child: SingleChildScrollView(
                  padding: const EdgeInsets.all(24),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.stretch,
                    children: [
                      const Icon(
                        Icons.sos,
                        size: 75,
                        color: Colors.deepPurple,
                      ),
                      const SizedBox(height: 20),
                      const Text(
                        'Welcome to Disaster Alert',
                        textAlign: TextAlign.center,
                        style: TextStyle(
                          fontSize: 23,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      const SizedBox(height: 25),

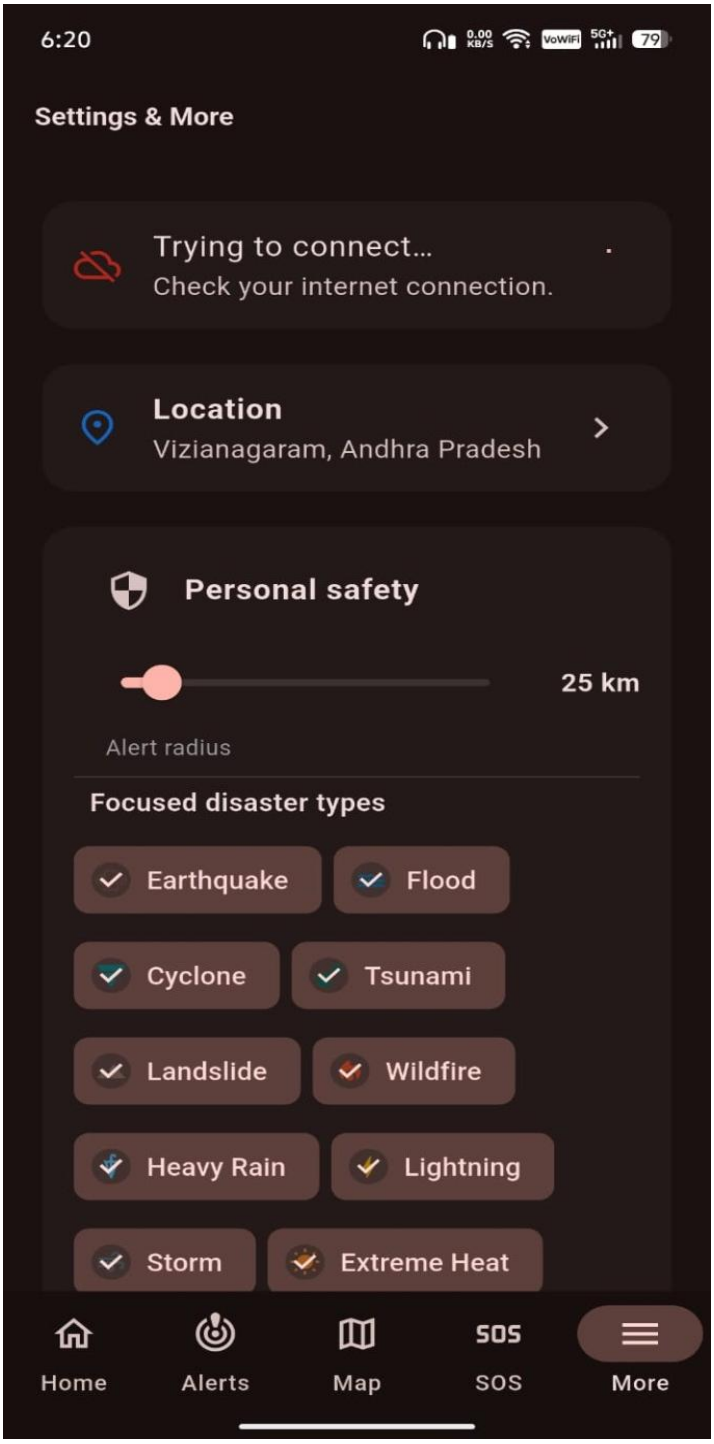

                      const SizedBox(height: 16),

                      if (_error != null)
                        Padding(
                          padding: const EdgeInsets.only(bottom: 12),
                          child: Text(
                            _error!,
                            style: const TextStyle(color: Colors.red),
                          ),
                        ),

                                  const SizedBox(height: 12),

                      CustomButton(
                        text: 'Create an Account',
                        icon: Icons.person_add,
                        color: Colors.grey,
                        onPressed: () {
                          Navigator.push(
                            context,
                            MaterialPageRoute(
                              builder: (_) => const NewContactScreen(),
                            ),
                          );
                        },
                      ),
                    ],
                  ),
                ),
              ),
            );
          }
        }
```

**Output:**




**Safety Settings Screen**

# 16 Animations

**Objective:** Add basic UI animations.

**Relation to Project:** Used for smooth transitions between screens.

**Description:**

Your Flutter project incorporates animations primarily in the SurvivalGuideScreen to enhance user interaction and provide a more engaging experience. The an-imations include:

* **Avatar Bounce Animation:**
  - Implemented using AnimationController and Tween in combination with AnimatedBuilder.
  - Creates a smooth vertical bouncing effect for the user avatar on the Survival Guide.
  - Provides visual feedback that the avatar is interactive, indicating that it can be tapped to edit the contact.
* **Option Cards Fade and Scale Animation:**
  - Implemented in the _buildOption() method using TweenAnimationBuilder.
  - Option cards scale from 0.8 to 1.0 and fade in with opacity animation, giving a smooth entrance effect.
  - Each card can have a slight delay (delay parameter) for a staggered animation effect.
* **Implicit Animations for UI Feedback:**
  - CircularProgressIndicator in New Contact and emergency-contact workflows pro-vides a smooth, rotating animation during asynchronous operations. in SOS and New Contact screens pro-vides a smooth, rotating animation during asynchronous operations.
  - While not manually animated, the progress indicators give a visual cue for loading states.

* **Navigation Transitions:**
  - Default MaterialPageRoute transitions are used for screen naviga-tion.
  - Provides smooth sliding animations when moving between screens such as Survival Guide → Live Alerts or Survival Guide → Home/Live Alerts-/Edit Contact.

**Animations:**

```
class _ResultScreenState extends State<ResultScreen>
    with SingleTickerProviderStateMixin {
  late ConfettiController _confettiController;
  late AnimationController _controller;
  late Animation<double> _fade;
  late Animation<double> _scale;
  late Animation<Offset> _contentSlide;
  late Animation<Offset> _buttonSlide;

  String _message = '';
  Color _messageColor = Colors.grey;
  bool _shouldPlayConfetti = false;
  @override
  void initState() {
    super.initState();
    _confettiController = ConfettiController(
      duration: const Duration(seconds: 3),
    );
    _controller = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 900),
    );
    _fade = Tween<double>(
      begin: 0.0,
      end: 1.0,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.easeIn,
      ),
    );
    _scale = Tween<double>(
      begin: 0.92,
      end: 1.0,
    ).animate(
      CurvedAnimation(
        parent: _controller,
        curve: Curves.elasticOut,
      ),
    );
    _contentSlide = Tween<Offset>(
      begin: const Offset(0, 0.15),
      end: Offset.zero,
    ).
```

```
          animate(
                CurvedAnimation(
                  parent: _controller,
                  curve: Curves.easeOut,
                ),
              );
          buttonSlide = Tween<Offset>(
                begin: const Offset(0, 0.35),
                end: Offset.zero,
              ).animate(
                CurvedAnimation(
                  parent: _controller,
                  curve: const Interval(
                    0.4,
                    1.0,
                    curve: Curves.easeOut,
                  ),
                ),
              );

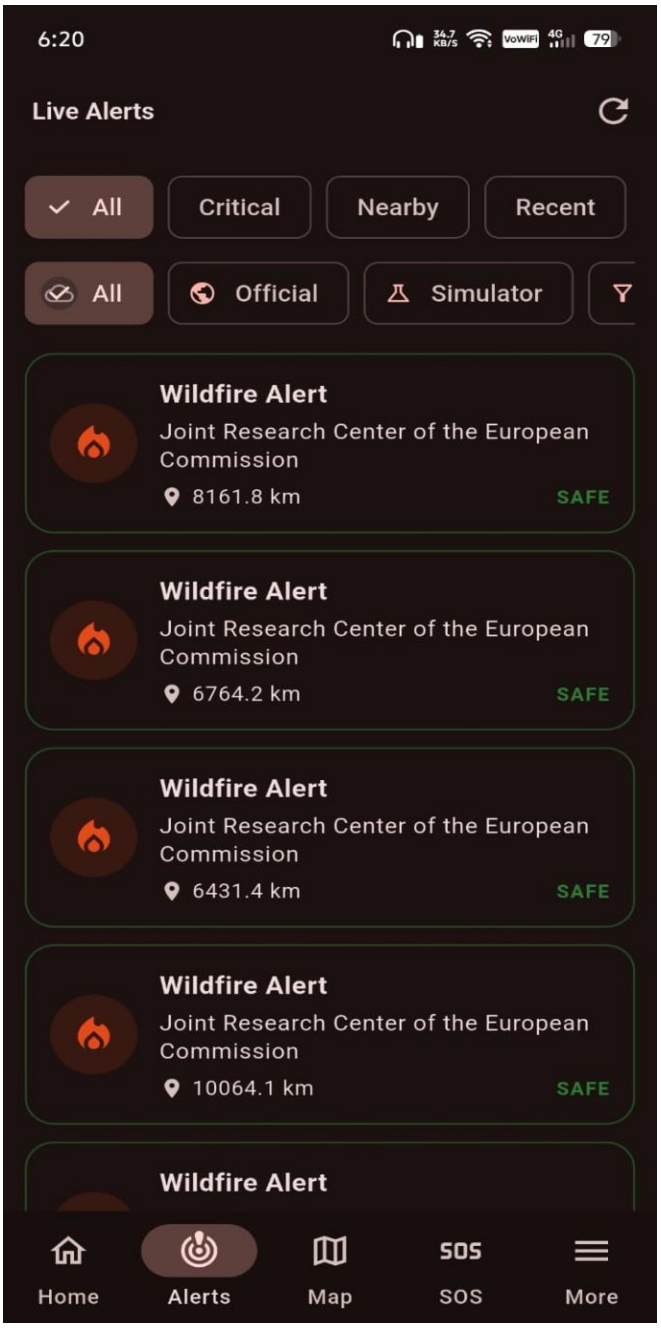
              // Emergency Contact SOS action result message
              _message = 'Your emergency contact SOS action was
    submitted!';
              _messageColor = Colors.green;
              _shouldPlayConfetti = true;

              _controller.forward();

              if (_shouldPlayConfetti) {
                _confettiController.play();
              }
            }

            @override
            void dispose() {
              _confettiController.dispose();
              _controller.dispose();
              super.dispose();
            }
          }
```

**Output:**



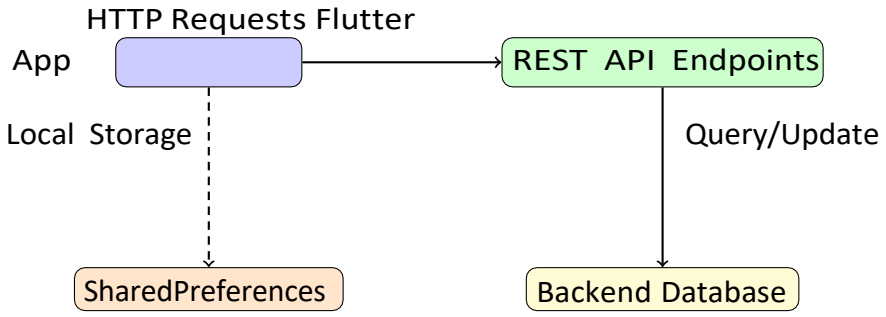

**Live Alerts Screen**

# 17 REST API Integration

**Objective:** Fetch data from API and display it.

**Relation to Project: Used in live_alerts to fetch user alert records.**

```
final response = await http . get( Uri. parse (' https :// api.
example . com / live_alerts '));
```




# 18 Testing and Debugging

**Objective:** To perform UI tests, unit tests, and use debugging tools to ensure the app functions correctly.

**Explanation:**

Testing is the process of verifying that the app works as intended. In this Flutter disaster app project, testing ensures that users can log in, attempt disaster events, view live alerts, and edit contacts without errors. Types of testing include:

* **Unit Testing:** Tests individual functions or classes.

  *Example: Checking if DatabaseHelper.getContacts() returns a valid token.*

* **Widget Testing:** Verifies that UI components display correctly and re-spond to user interactions.

  *Example: Checking if the SosScreen shows a "SOS" button.*

* **Integration Testing:** Tests end-to-end flow, including REST API calls.

  *Example: Open Survival Guide → View Live Alerts → Open Alert Details → Send SOS.*

* **Manual Testing:** Running the app on a device or emulator to test user interactions directly.

Debugging involves identifying and fixing errors in UI, API calls, or local storage. Tools used include Flutter DevTools, print/log statements, try/catch blocks, and Postman for API testing.

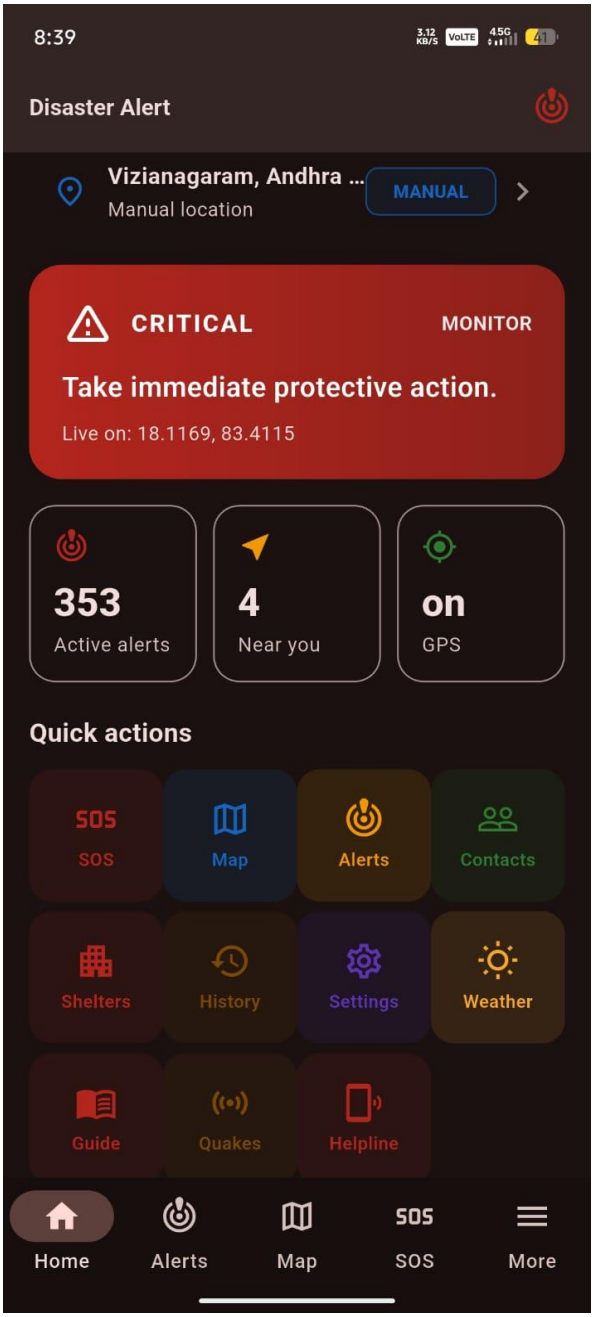
**Sample UI Test Code:**

```
testWidgets('Login Screen loads', (
  tester) async { await     tester. pump
  Widget( MyApp ()); expect(find.text('
  Login'), findsOneWidget);
});
```

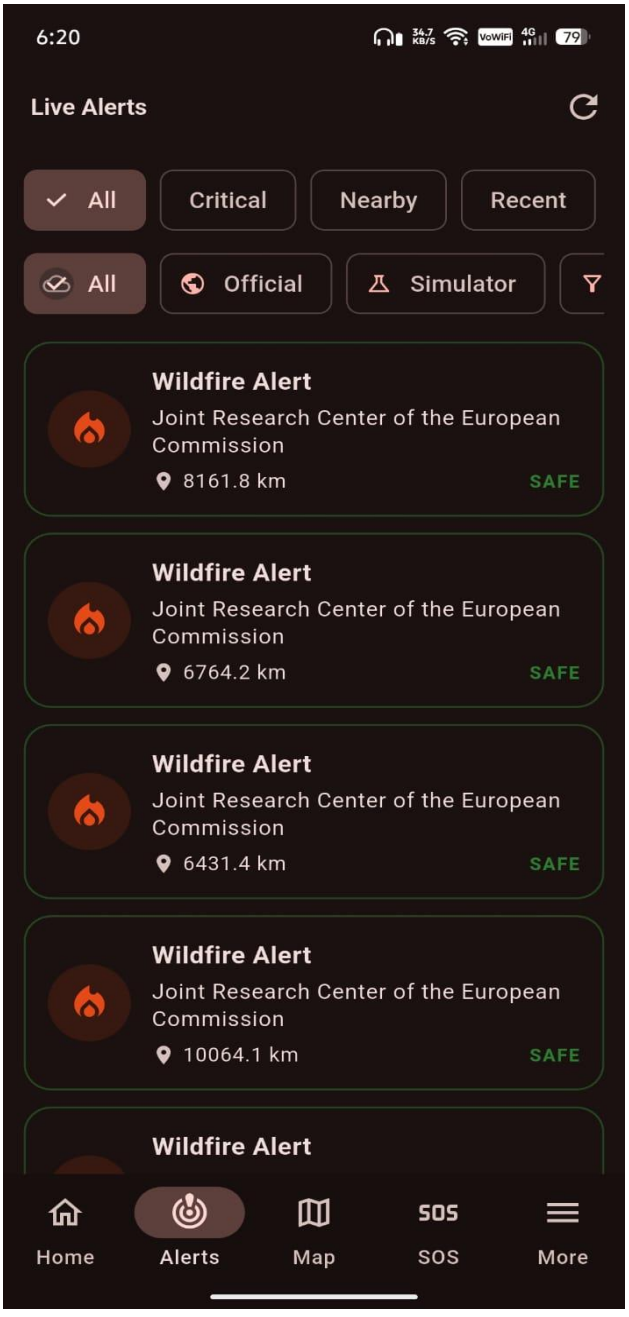
**Sample Debugging Example:**

```
try {
  // Check that local SQLite operations return the expected result.
    print(' SOS failed : Invalid response ');
  }
} catch (e) {
  print(' SOS error: $e ');
}
```

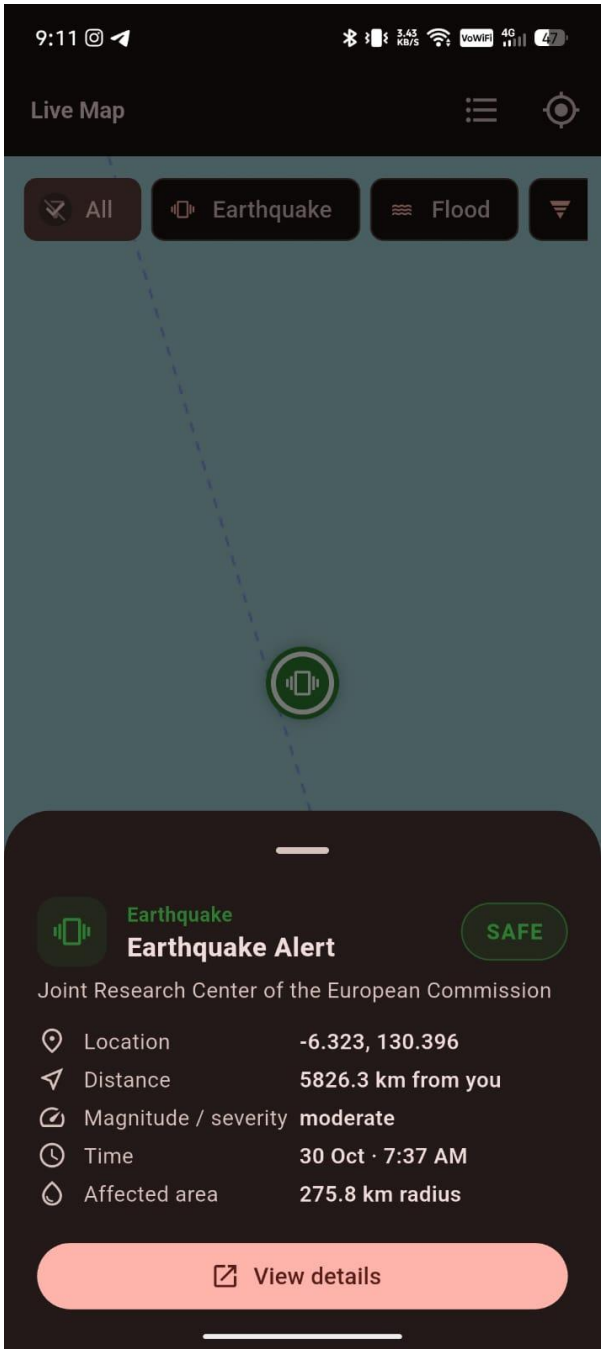
# 19 Work Flow Of Screens:



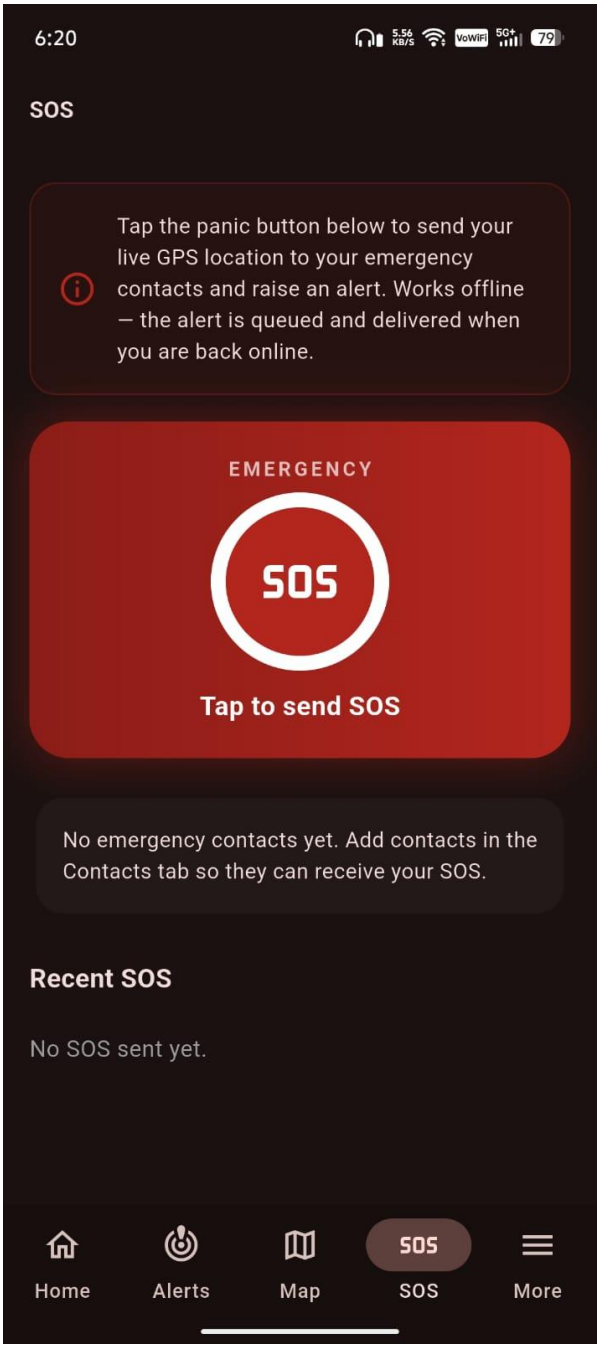

Home Screen



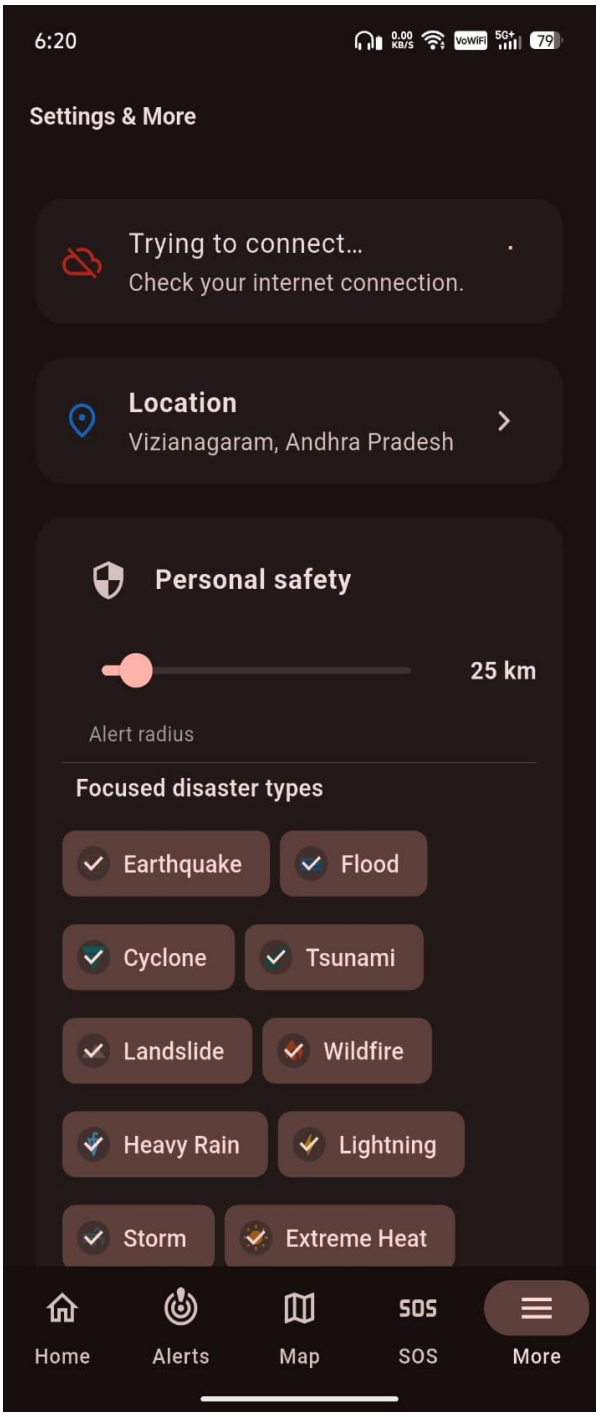

Alerts Screen




Map Screen



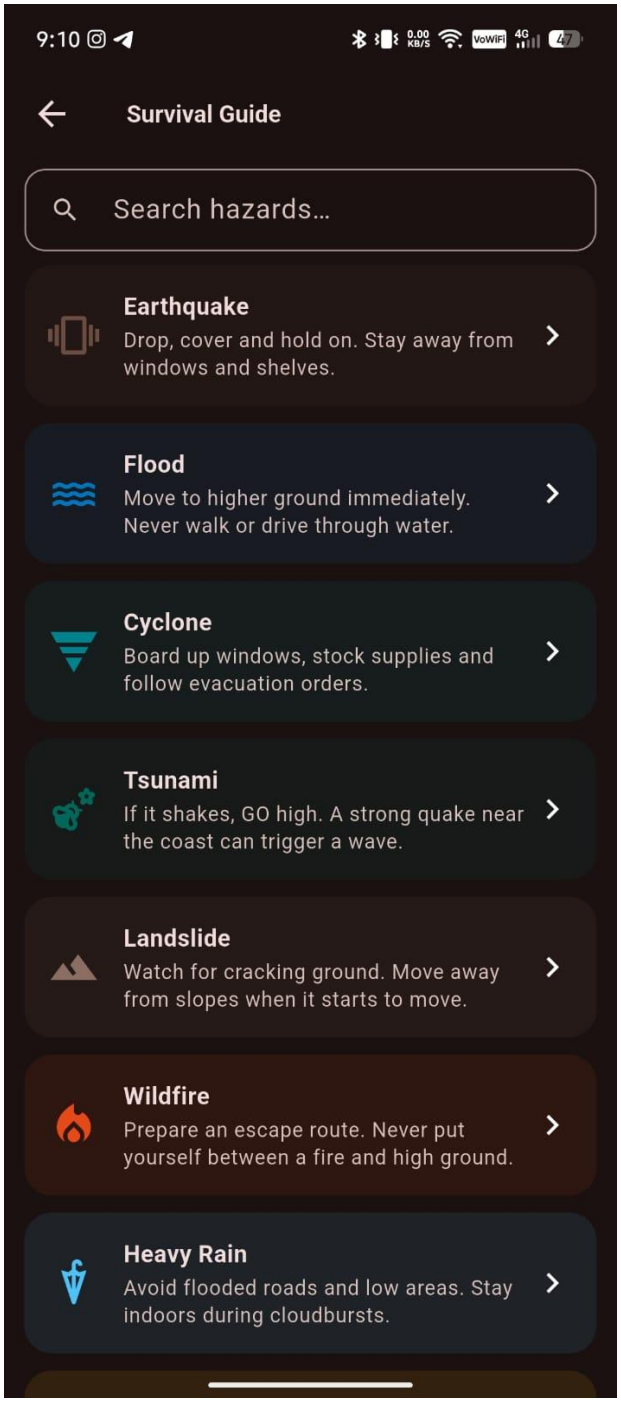

SOS Screen



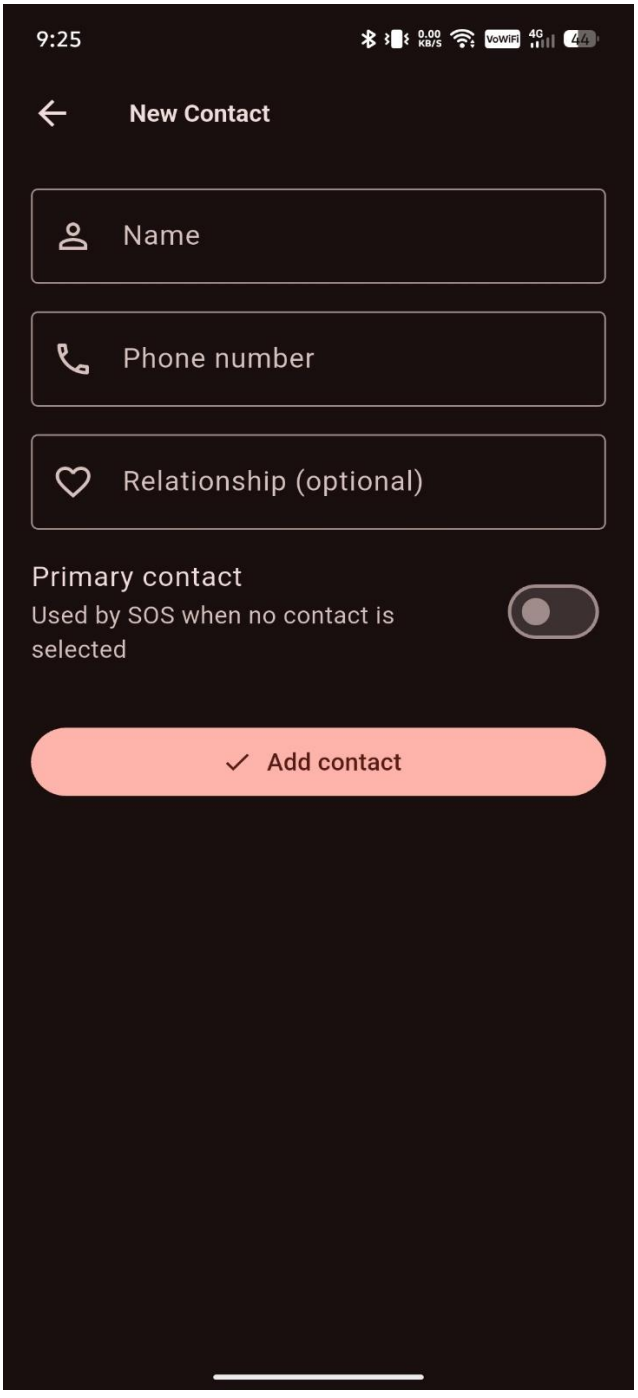

Settings Screen




Weather Screen




Survival Guide Screen




Emergency Contact Screen

# 20 Conclusion

Through this project, core Flutter principles such as widgets, layouts, navigation, state management, and responsive design were applied to a disaster-support mobile application. The Disaster Alert App brings together hazard guidance, weather information, live alert browsing, map-based alert inspection, and emergency actions in a single interface.

The project also demonstrates local SQLite data storage for emergency contacts and related records, reusable interface components, and screen-to-screen navigation. The supplied application screenshots illustrate the Survival Guide, Earthquake Guide, Weather, Settings, SOS, Live Map, Live Alerts, Alert Details, and New Contact screens. The report adapts the reference Flutter examples to these project features; code snippets are illustrative adaptations rather than verified extracts from the application source code.